Advanced Machine Learning, ETH Zurich

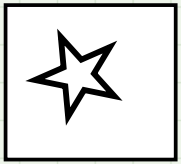
Convolutional neural networks

Carlos Cotrini

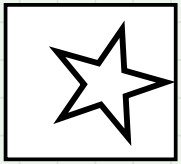
Autumn semester 2026

Entry 1

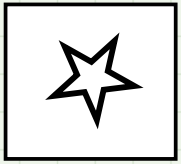
Stars and planets



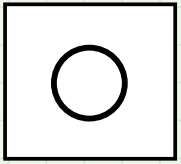
Star



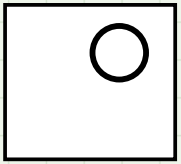
Star



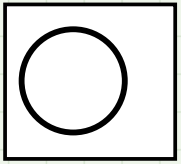
Star



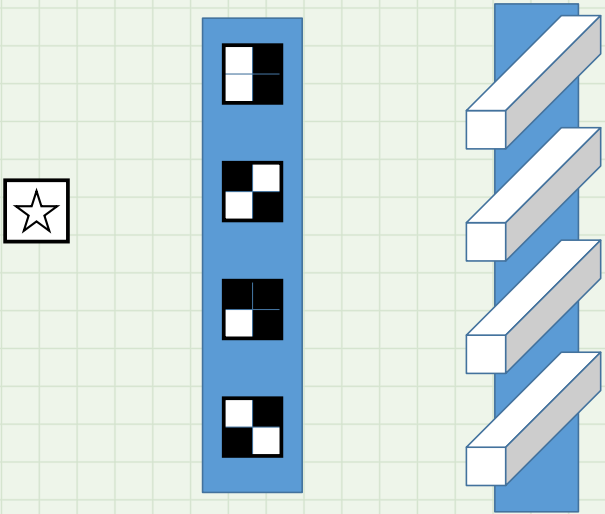
Planet



Planet

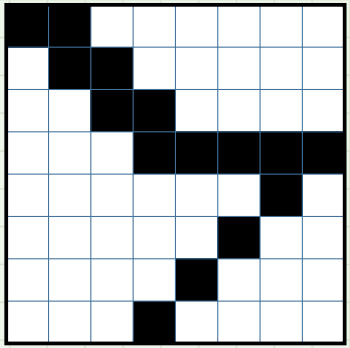


Planet



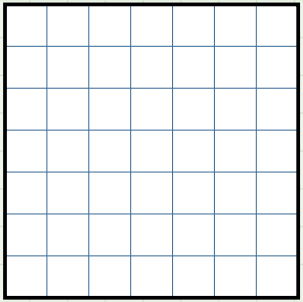
Entry 2

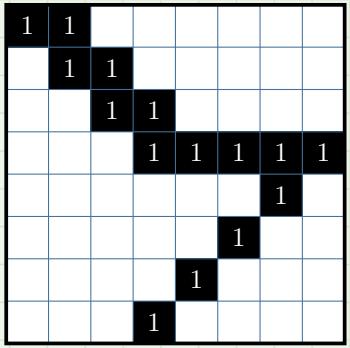
Convolutional filters





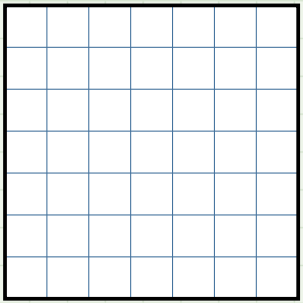
Convolutional filter

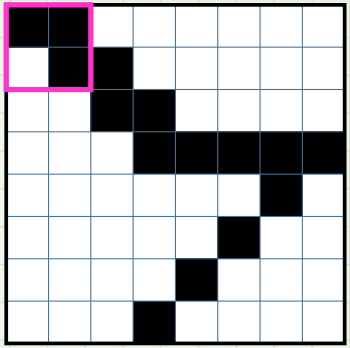






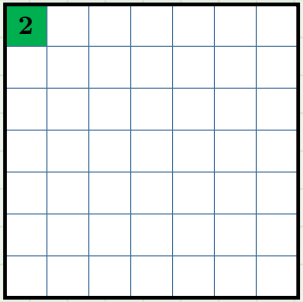
Convolutional filter

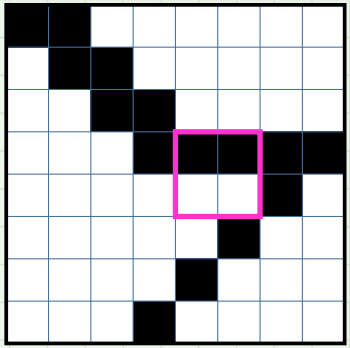






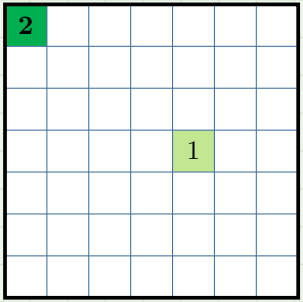
Convolutional filter

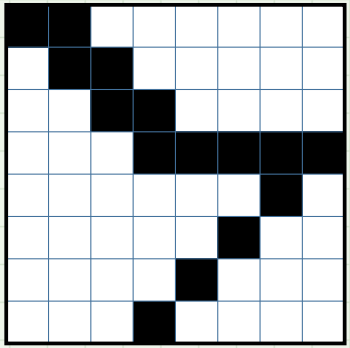






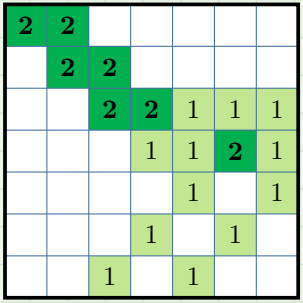
Convolutional filter

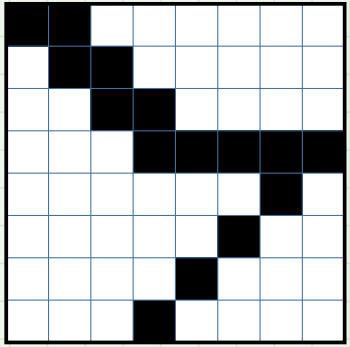






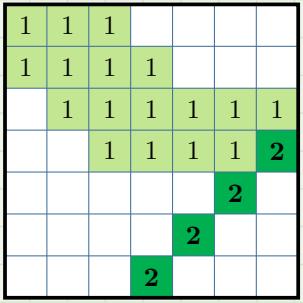
Convolutional filter







Convolutional filter



1	3	5
1	2	4
6	3	2

Input

1	2
−5	0

ReLU

Convolutional filter

Output

1	3	5
1	2	4
6	3	2

Input

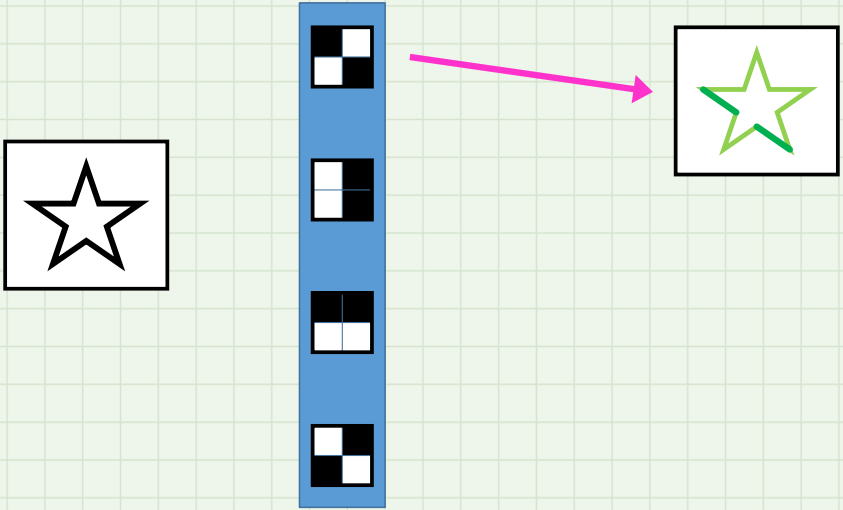
1	2
−5	0

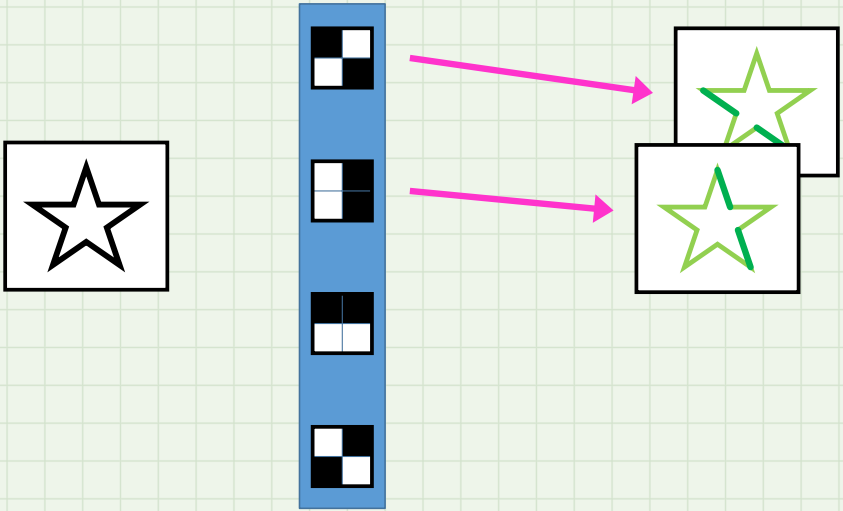
ReLU

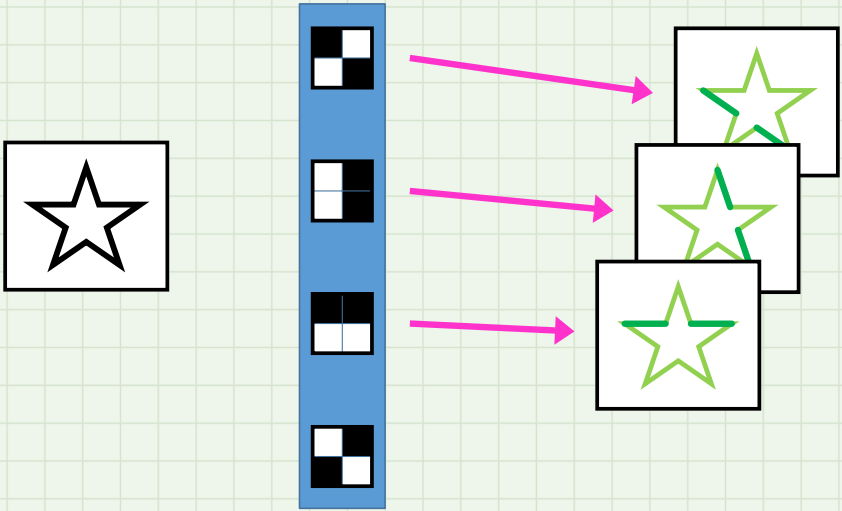
Convolutional filter

2	3
0	0

Output

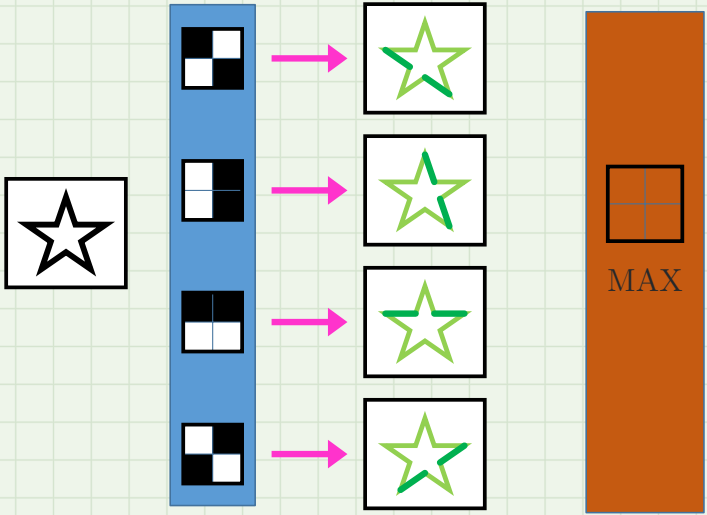


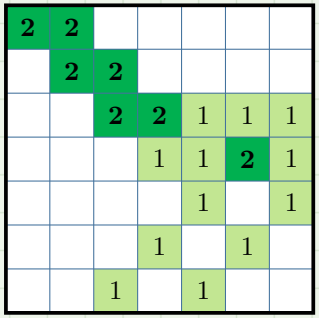




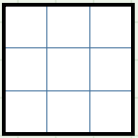
Entry 3

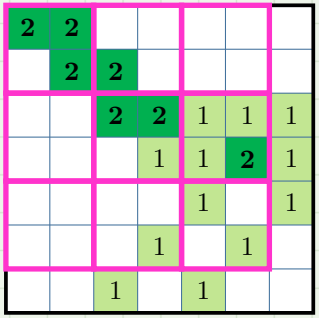
Pooling

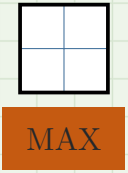

MAX

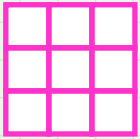

2 2
2 2
2 2 1 1 1
1 1 2 1
1 1
1 1
1 1

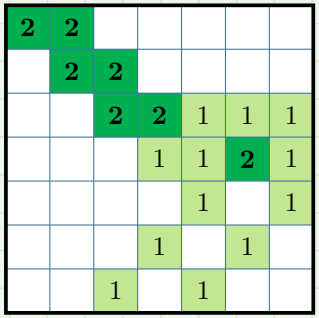

MAX



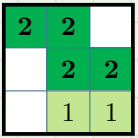

2 2
2 2
2 2 1 1 1
1 1 2 1
1 1
1 1
1 1

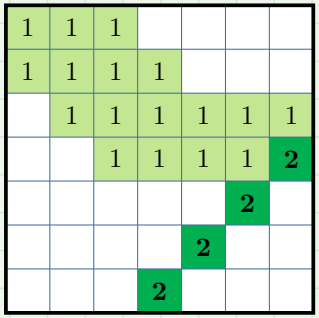

MAX

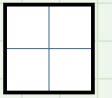



2 2
2 2
2 2 1 1 1
1 1 2 1
1 1
1 1
1 1

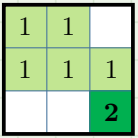

MAX


2 2
2 2
1 1



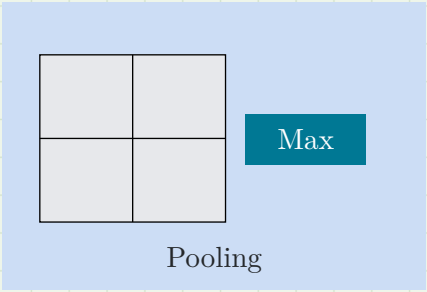


MAX



1	0	5	3	1
4	0	0	2	2
6	1	0	1	0
0	0	0	2	4
3	0	1	2	1

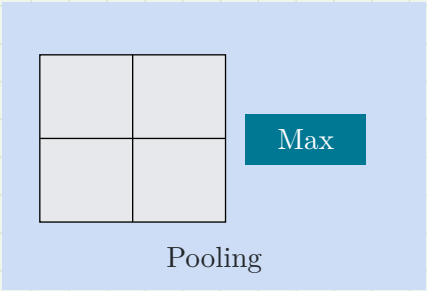
Input



Output

1	0	5	3	1
4	0	0	2	2
6	1	0	1	0
0	0	0	2	4
3	0	1	2	1

Input

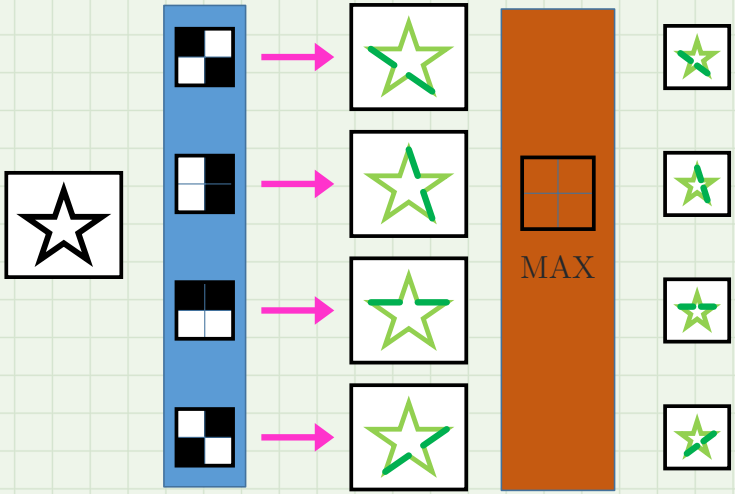


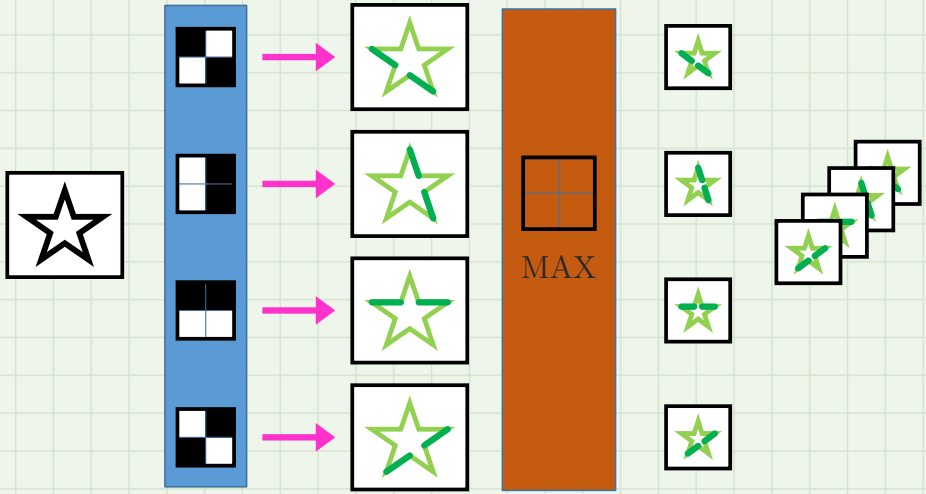
4	5
6	2

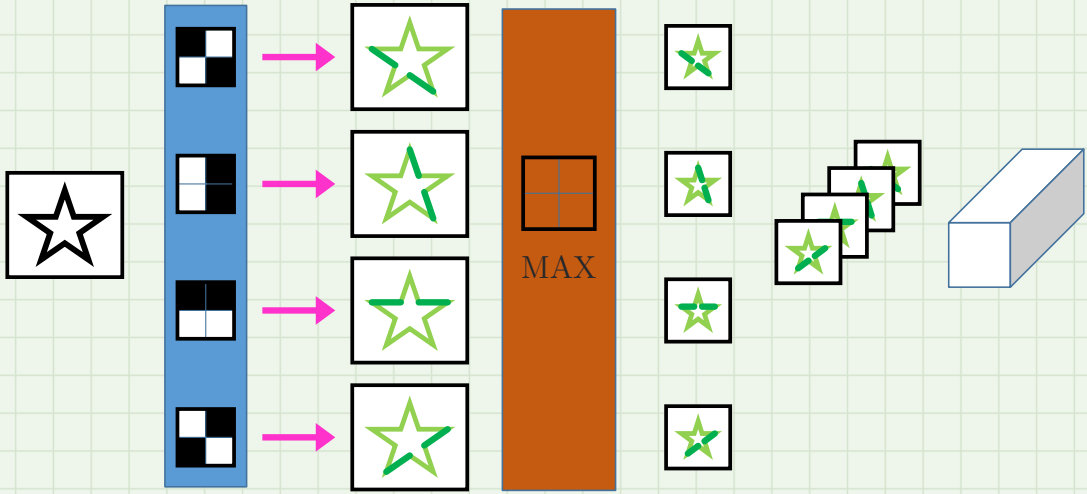
Output

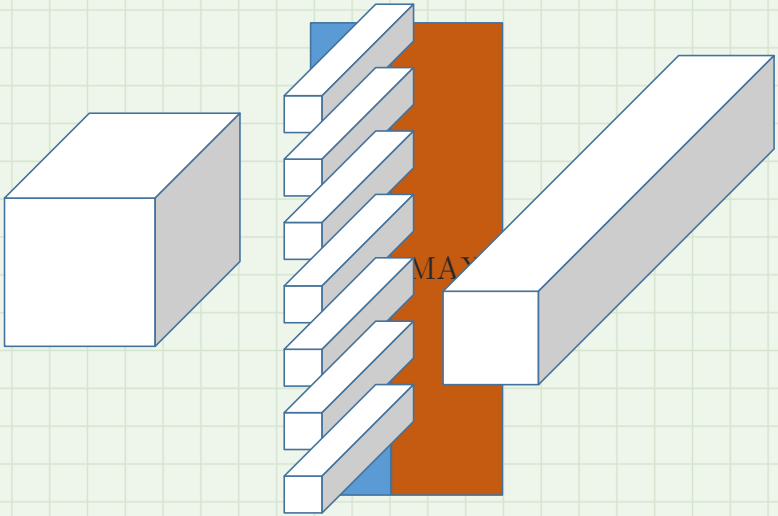
Entry 4

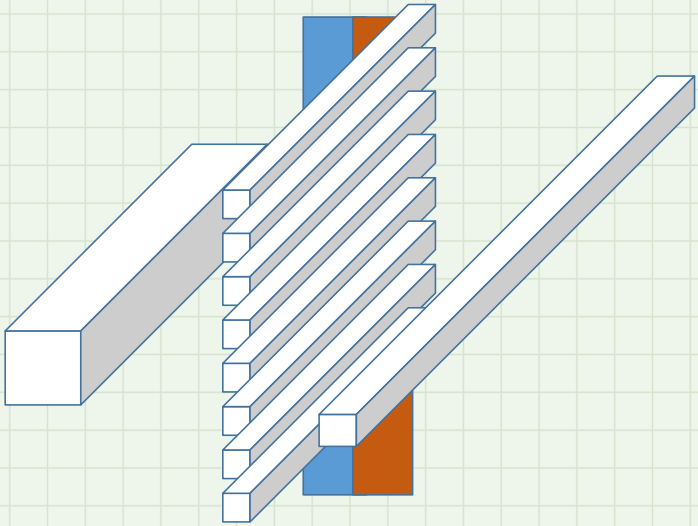
The network

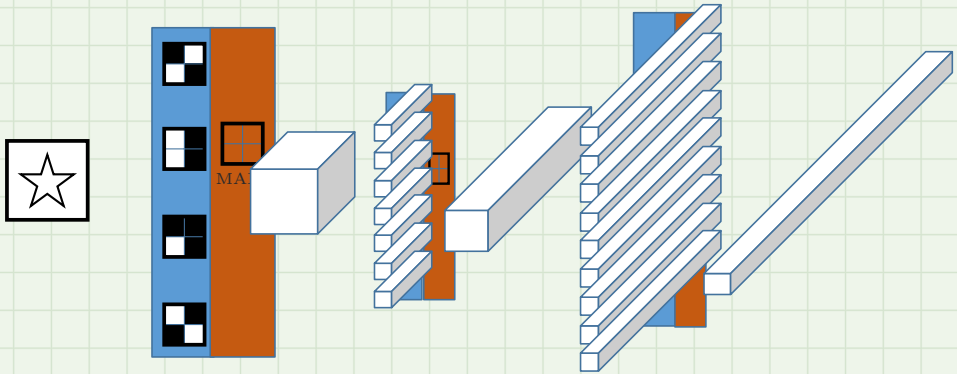

MAX

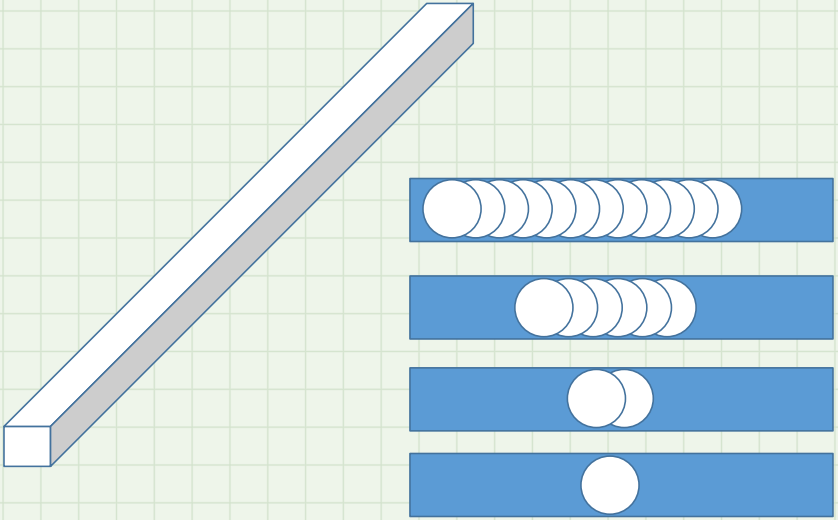

MAX

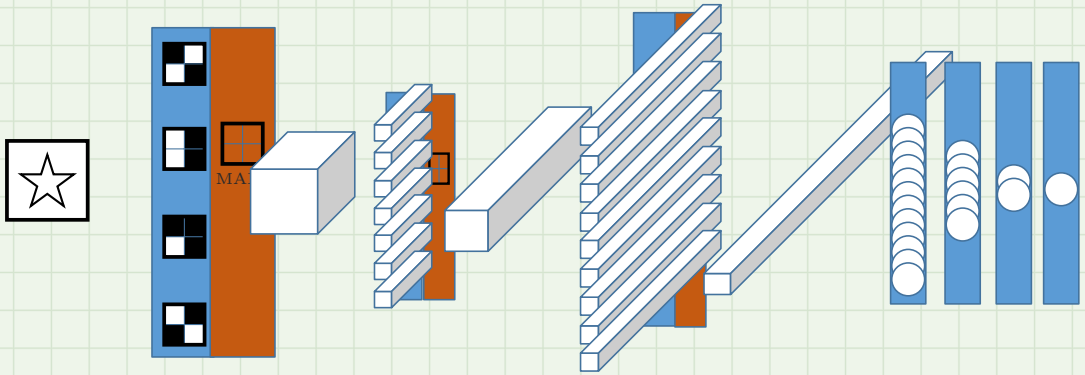

MAX


MAX



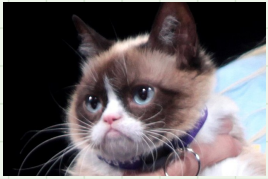




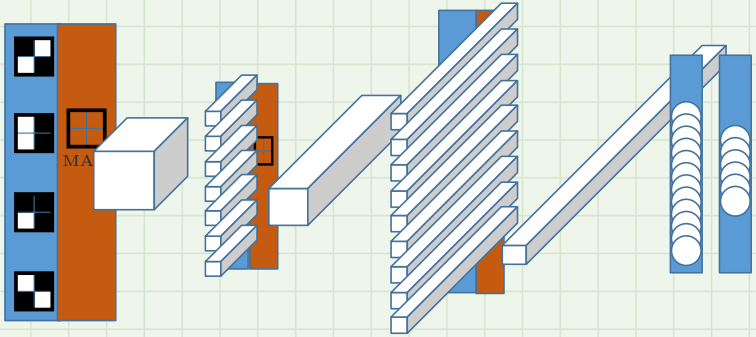


Entry 5

Image recognition and softmax

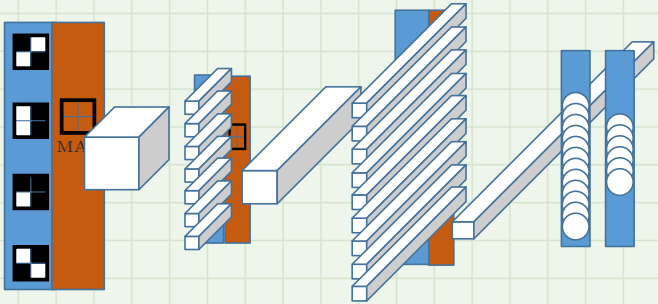


Grumpy Cat: Gage Skidmore, CC BY-SA 3.0, Wikimedia Commons.





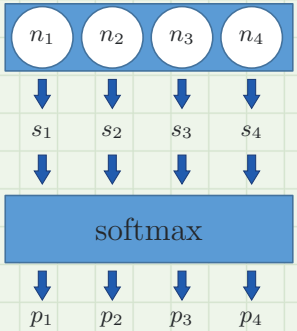
Rabbit: JM Ligero Loarte, CC BY 3.0. Cat: Alvesgaspar, CC BY-SA 3.0. Dog: Dog breed facts, CC BY-SA 4.0. All from Wikimedia Commons.





Rabbit: JM Ligero Loarte, CC BY 3.0. Cat: Alvesgaspar, CC BY-SA 3.0. Dog: Dog breed facts, CC BY-SA 4.0. All from Wikimedia Commons.

- In binary classification tasks, we want to recognize if a particular given object belongs to one of two classes. For these tasks, the CNN has one neuron at the end, whose activation function is the sigmoid function. It produces a value between 0 and 1, indicating the probability that the input belongs to one particular class.
- In multi-class classification tasks, we want to recognize if a particular given object belongs to one of K classes. For these tasks, the CNN has K neurons $n_1, n_2, \ldots, n_K$ at the end, each of them producing a score of how much each neuron believes that the object belongs to one of these classes. After these K neurons, we have a softmax function that transforms these values into probabilities $p_1, p_2, \ldots, p_K$. Each probability p_i indicates the probability that the object belongs to class i.



Entry 6

Image classification with a CNN

Our example is CIFAR-10: 50,000 training images of 32×32 pixels in 3 colours, each showing one of 10 objects.



CIFAR-10: A. Krizhevsky, Learning multiple layers of features from tiny images, 2009.

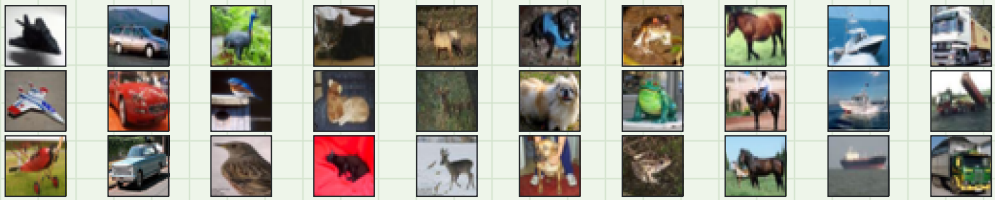
Our example is CIFAR-10: 50,000 training images of 32×32 pixels in 3 colours, each showing one of 10 objects.



Let $f : \underset{\text{images}}{\mathcal{X}} \to \underset{\text{objects}}{\mathcal{Y}}$ and $(x, y) \sim p^*$ Unknown

CIFAR-10: A. Krizhevsky, Learning multiple layers of features from tiny images, 2009.

Our example is CIFAR-10: 50,000 training images of 32×32 pixels in 3 colours, each showing one of 10 objects.

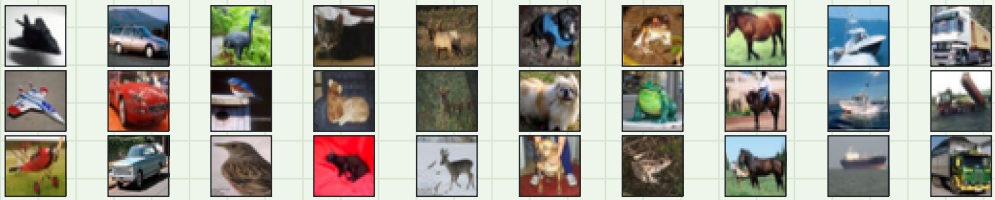


Let $f : \underset{\text{images}}{\mathcal{X}} \to \underset{\text{objects}}{\mathcal{Y}}$ and $(x, y) \sim p^*$ Unknown

Given $x_1, y_1, \ldots, x_n, y_n \sim p^*$

CIFAR-10: A. Krizhevsky, Learning multiple layers of features from tiny images, 2009.

Our example is CIFAR-10: 50,000 training images of 32×32 pixels in 3 colours, each showing one of 10 objects.



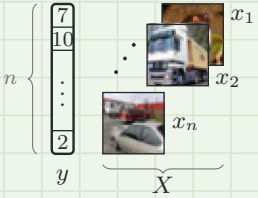
Let $f : \underset{\text{images}}{\mathcal{X}} \to \underset{\text{objects}}{\mathcal{Y}}$ and $(x, y) \sim p^*$ Unknown

Given $x_1, y_1, \ldots, x_n, y_n \sim p^*$

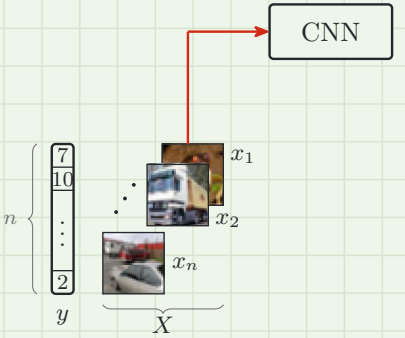
Find $\hat{f}$ such that $\hat{f} \approx f$

CIFAR-10: A. Krizhevsky, Learning multiple layers of features from tiny images, 2009.

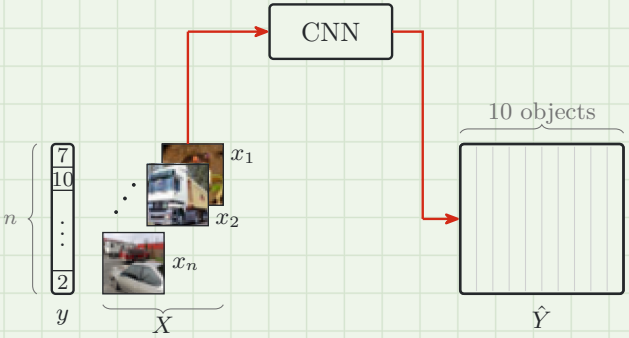
Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.



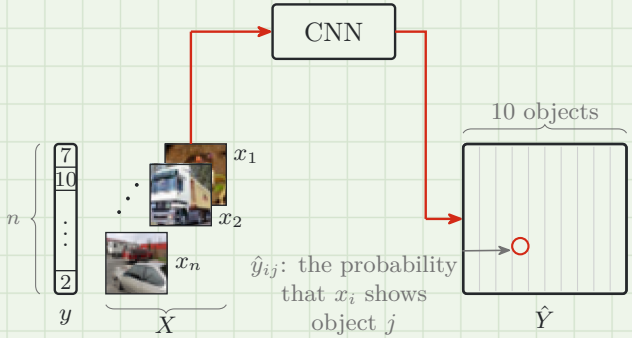
Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.



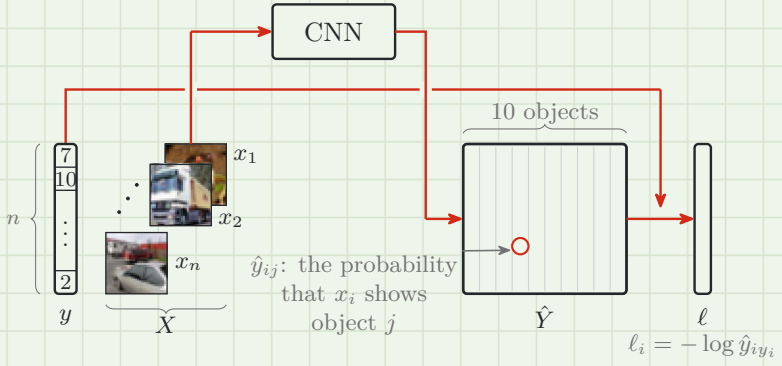
Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.



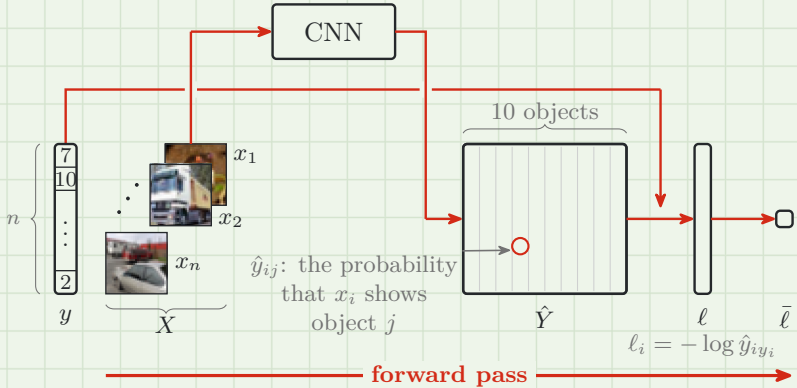
Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.



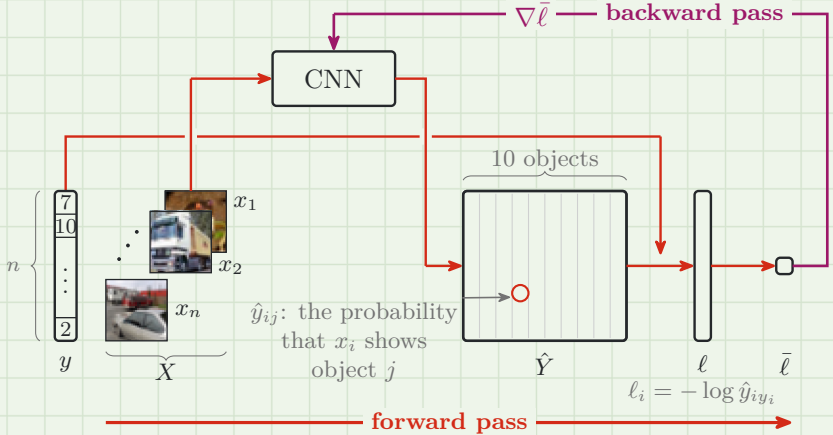
Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.



Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.



Each image x_i shows the object y_i, a number from 1 to 10. The CNN gives each image a row of $\hat{Y}$, and the losses compare the rows with y.

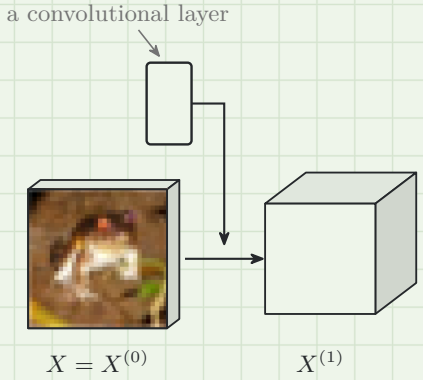


Each convolutional layer turns a box of numbers into the next box, with its own block of filters. We follow one image: $X^{(0)}$ is the image itself.

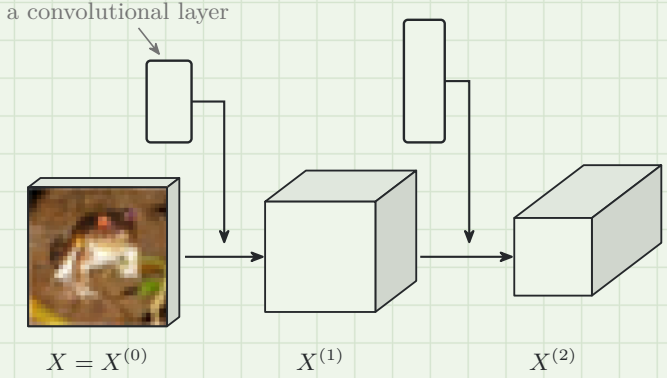


$X = X^{(0)}$

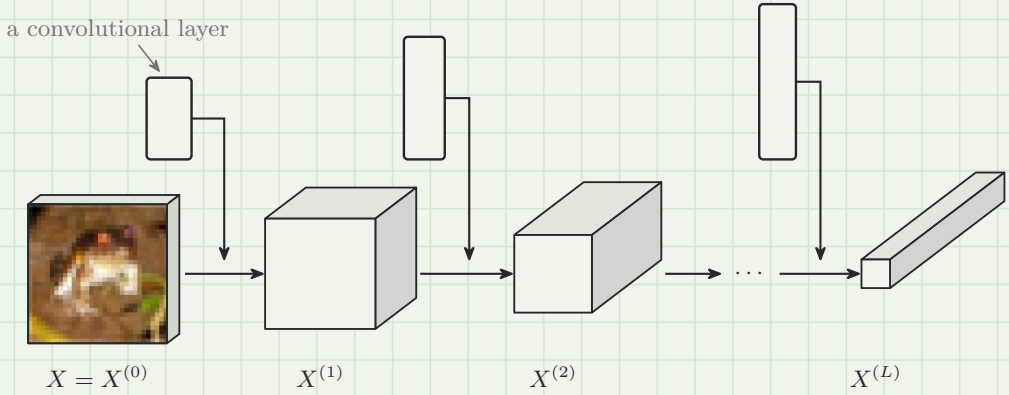
Each convolutional layer turns a box of numbers into the next box, with its own block of filters. We follow one image: $X^{(0)}$ is the image itself.



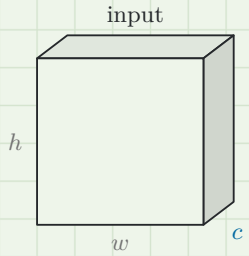
Each convolutional layer turns a box of numbers into the next box, with its own block of filters. We follow one image: $X^{(0)}$ is the image itself.



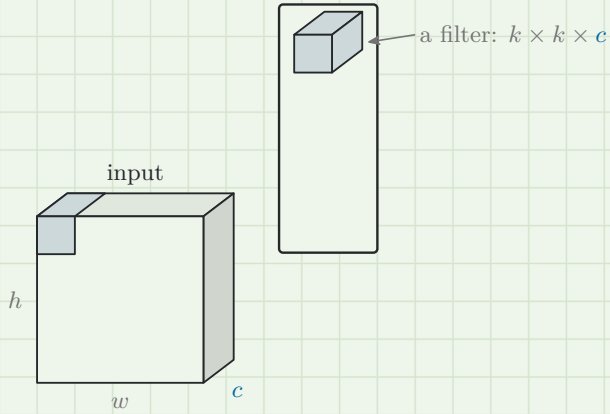
Each convolutional layer turns a box of numbers into the next box, with its own block of filters. We follow one image: $X^{(0)}$ is the image itself.



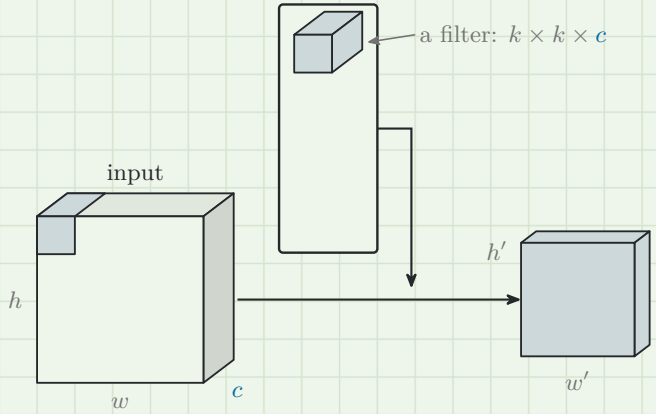
Each filter is as deep as the input. It slides over the input and gives one map, so d filters give an output with d channels.



Each filter is as deep as the input. It slides over the input and gives one map, so d filters give an output with d channels.

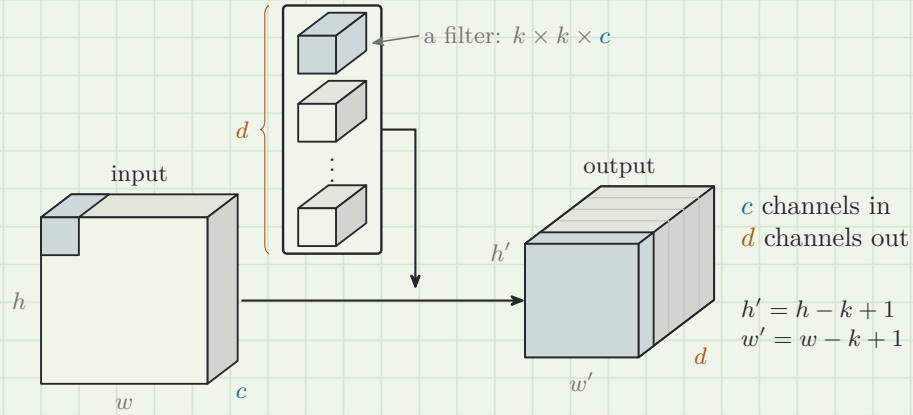


Each filter is as deep as the input. It slides over the input and gives one map, so d filters give an output with d channels.

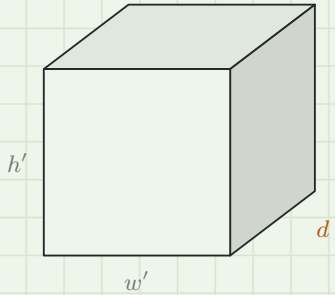


$$h' = h - k + 1$$
$$w' = w - k + 1$$

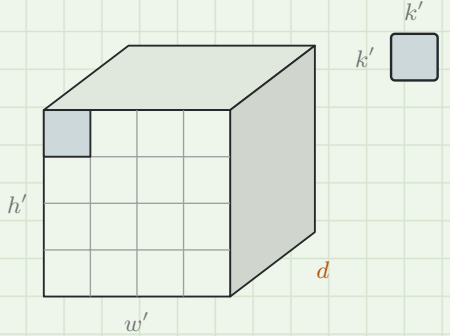
Each filter is as deep as the input. It slides over the input and gives one map, so d filters give an output with d channels.



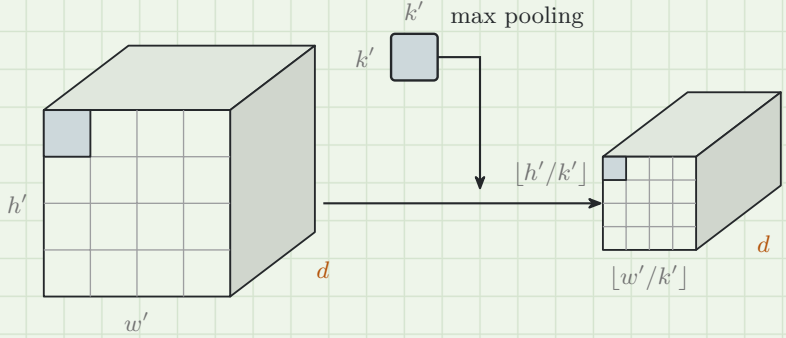
Pooling cuts each map into windows of $k' \times k'$ and keeps the largest number of each window. The height and the width shrink; the number of channels stays d.



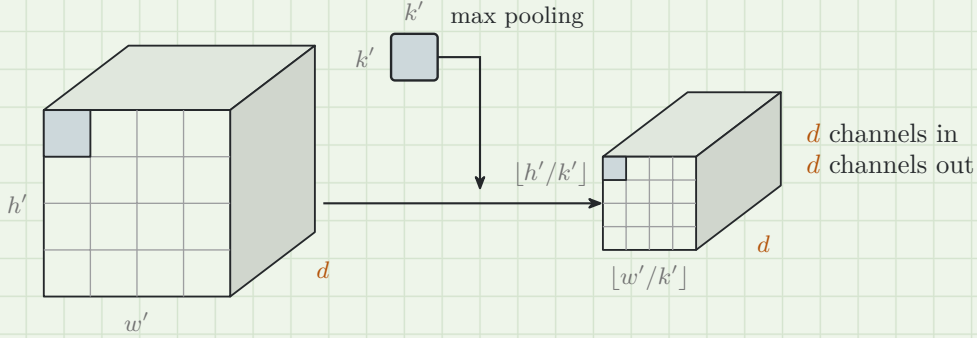
Pooling cuts each map into windows of $k' \times k'$ and keeps the largest number of each window. The height and the width shrink; the number of channels stays d.



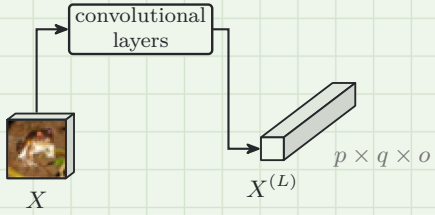
Pooling cuts each map into windows of $k' \times k'$ and keeps the largest number of each window. The height and the width shrink; the number of channels stays d.



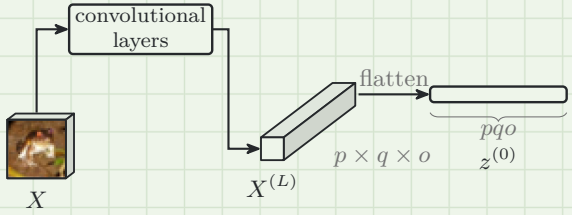
Pooling cuts each map into windows of $k' \times k'$ and keeps the largest number of each window. The height and the width shrink; the number of channels stays d.



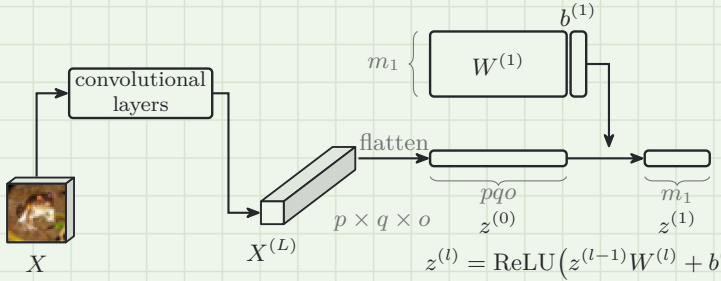
The box $X^{(L)}$ is flattened into one vector $z^{(0)}$. A standard neural network turns $z^{(0)}$ into $\hat{y}$, the row of $\hat{Y}$ of this image: its 10 probabilities.



The box $X^{(L)}$ is flattened into one vector $z^{(0)}$. A standard neural network turns $z^{(0)}$ into $\hat{y}$, the row of $\hat{Y}$ of this image: its 10 probabilities.

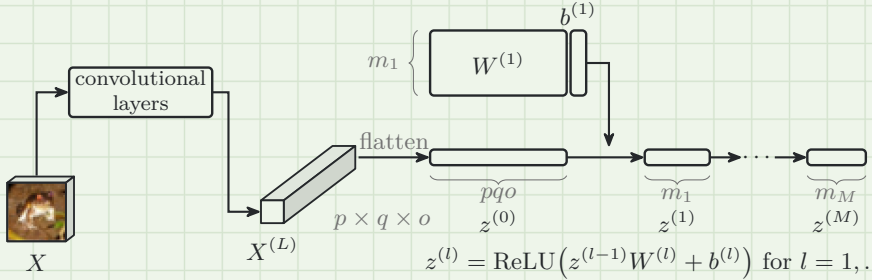


The box $X^{(L)}$ is flattened into one vector $z^{(0)}$. A standard neural network turns $z^{(0)}$ into $\hat{y}$, the row of $\hat{Y}$ of this image: its 10 probabilities.



$$z^{(l)} = \mathrm{ReLU}\left(z^{(l-1)}W^{(l)} + b^{(l)}\right) \text{ for } l = 1, \ldots, M$$

The box $X^{(L)}$ is flattened into one vector $z^{(0)}$. A standard neural network turns $z^{(0)}$ into $\hat{y}$, the row of $\hat{Y}$ of this image: its 10 probabilities.



$$z^{(l)} = \mathrm{ReLU}\left(z^{(l-1)}W^{(l)} + b^{(l)}\right) \text{ for } l = 1, \ldots, M$$

The box $X^{(L)}$ is flattened into one vector $z^{(0)}$. A standard neural network turns $z^{(0)}$ into $\hat{y}$, the row of $\hat{Y}$ of this image: its 10 probabilities.

